

Scaling the Municipal Revenue Platform to Multi-Tenant SaaS

TARGET MODEL	PRIMARY STACKS	INTEGRATIONS	TARGET AUDIENCE
Multi-Tenant SaaS	Next.js / Prisma / Supabase	Paystack & Arkesel APIs	Revenue Contractors / Assemblies

Executive Objective: Transform the single-assembly municipal back-office tool into an autonomous, self-service SaaS platform. Contractors awarded collection contracts can onboard instantly, configure bank settlements via Paystack API, manage custom Arkesel Sender IDs, and ingest raw ratepayer data via smart CSV mapping without administrative intervention.

1. DATABASE ARCHITECTURE & SCHEMA DELTA

To support multi-tenancy while strictly preserving data isolation, a top-level Tenant model is introduced in Prisma. All operational data (properties, transactions, SMS dispatches, GCRs) are scoped to a tenantId.

ISOLATION LAYER Prisma / RLS Tenant-scoped queries & Supabase Row Level Security	PAYMENT ROUTING Paystack Subaccounts Automated dynamic splitting at transaction initialization	SMS DISPATCH Dynamic Sender ID Tenant branding with generic system fallback
--	--	---

1.1 Top-Level Tenant Model Definition

```
// Modern Prisma Multi-Tenant Extension (schema.prisma)
model Tenant {
  id                String          @id @default(uuid())
  companyName       String
  assemblyName      String          // e.g. "KKMA", "Ga East", "STMA"

  // Paystack Autonomous Settlement Configuration
  paystackSubaccountCode String?    // Stores Paystack "ACCT_XXXXXX"
  bankCode          String?        // Settlement Bank Code (e.g. "057")
  accountNumber      String?        // Settlement Account Number
  accountName        String?        // Resolved Bank Account Name

  // Arkesel Dynamic Communication Configuration
  customSenderId     String?        // Requested Sender ID (max 11 chars)
  senderIdStatus     String          @default("PENDING") // PENDING | APPROVED | REJECTED

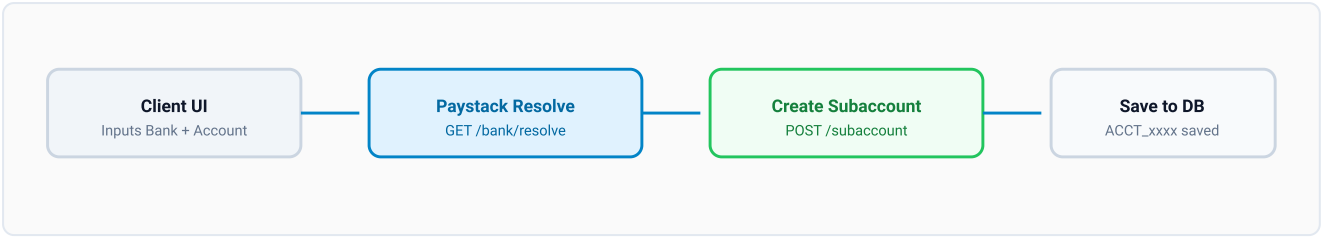
  createdAt          DateTime       @default(now())
  updatedAt          DateTime       @updatedAt

  // Tenant-Scoped Relational Cascades
  adminUsers         AdminUser[]
  properties          Property[]
  transactions        Transaction[]
  smsJobs             SmsRolloutJob[]
  collectors          Collector[]
}
```

1.2 Relational Scoping Delta Across Core Models

2. AUTONOMOUS PAYSTACK SETTLEMENT ENGINE

Clients configure their payout destinations directly in their settings. The platform programmatically provisions Paystack subaccounts, eliminating manual setup while allowing optional platform commission splits.



2.1 Paystack API Orchestration Flow

- **Account Verification:** Front-end queries Paystack's bank resolution endpoint to confirm business name before subaccount creation.
- **Automated Provisioning:** Platform issues a server-side POST /subaccount payload attaching settlement bank code, account number, and percentage charge (platform fee).
- **Dynamic Ratepayer Checkout:** During bill checkout initialization, the system injects the contractor's stored paystackSubaccountCode into the payload:

```
// Dynamic Paystack Transaction Initialization Payload
const paystackPayload = {
  amount: bill.totalAmountDue * 100, // Amount in pesewas
  email: ratepayer.email || "ratepayer@assembly.gov.gh",
  reference: `TX-${Date.now()}-${bill.accountNumber}`,
  subaccount: tenant.paystackSubaccountCode, // Directs settlement to contractor
  bearer: "subaccount", // Contractor absorbs transaction fee
  metadata: { tenantId: tenant.id, propertyId: property.id }
};
```

3. AUTONOMOUS ARKESEL SMS ENGINE & FALLBACK ROUTING

Because telecom operator approval for custom Sender IDs can take several business days, the SMS engine enforces an automated status machine and dynamic fallback router to ensure dispatches are never blocked.

Sender ID Status	Active Dispatch Header	System Routing Behavior
APPROVED	Client Sender ID (e.g. K-COLLECT)	Dispatches directly using contractor's verified alphanumeric header.
PENDING	REVENUE-MSG (Generic System Header)	Reroutes via default verified platform header + appends assembly signature in body.
REJECTED	REVENUE-MSG (Generic System Header)	Alerts contractor UI to request alternative Sender ID while maintaining dispatches.

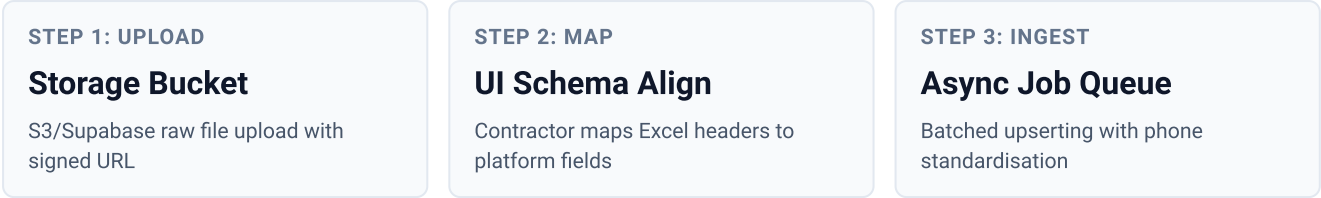
3.1 Dynamic SMS Routing Logic

```
// Autonomous Sender ID Selection Logic in SMS Engine
async function resolveSenderId(tenantId: string): Promise {
  {
    const tenant = await db.tenant.findUnique({ where: { id: tenantId } });

    if (tenant?.senderIdStatus === "APPROVED" && tenant.customSenderId) {
      return tenant.customSenderId; // Use verified contractor header
    }
  }
}
```

4. FRICTIONLESS CSV INGESTION & FIELD MAPPING ENGINE

Municipal assemblies issue ratepayer spreadsheets with inconsistent headers. The client interface provides a dynamic column mapping UI, while processing is executed asynchronously via background workers.



4.1 Asynchronous Bulk Ingestion Pipeline

- **Chunked Processing:** Background workers parse CSV files in 500-record batches to prevent memory limits during massive multi-thousand ratepayer imports.
- **Automatic Phone Normalization:** Strips invalid characters and formats Ghanaian phone numbers (e.g., converting 024XXXXXXX to international format 23324XXXXXXX).
- **Upsert Strategy:** Uses composite keys [accountNumber, tenantId] to update existing balances without creating duplicate ratepayer entries.

5. IMPLEMENTATION PHASE & SCALING CHECKLIST

A structured execution roadmap to transition the platform from single-assembly deployment to multi-tenant production readiness.

Phase	Milestone Deliverables	Technical Focus
Phase 1: Database Migration	• Deploy Tenant schema update. • Backfill existing KKMA records to initial Tenant record. • Enable Supabase Row-Level Security policies.	Prisma Migrations & RLS Policy Enforcement
Phase 2: Self-Service Integrations	• Build Bank Resolution & Subaccount UI. • Connect Paystack Subaccount API endpoints. • Build Arkesel Sender ID request & status polling UI.	Paystack & Arkesel Webhooks Integration
Phase 3: Ingestion & Messaging Queue	• Implement drag-and-drop CSV column mapping UI. • Upgrade SmsRolloutJob queue with background workers. • Add progress polling & real-time delivery dashboards.	Supabase Edge Functions / Async Queue Architecture

Next Steps: Review this architectural blueprint with your development team. Phase 1 database migration can be executed immediately with zero downtime for current KKMA operations.